
ORACLE
Academy

Database Programming with PL/SQL

8-1

Creating Procedures

ORACLE
Academy

Objectives

- This lesson covers the following objectives:
  - Differentiate between anonymous blocks and subprograms
  - Identify the benefits of subprograms
  - Define a stored procedure
  - Create a procedure
  - Describe how a stored procedure is invoked
  - List the development steps for creating a procedure
  - Create a nested subprogram in the declarative section of a procedure

Purpose

- There are times that you want to give a set of steps a name
- For example, if you're told to take notes, you know that this means you need to get out a piece of paper and a pencil and prepare to write
- So far you have learned to write and execute anonymous PL/SQL blocks (blocks that do not have a name associated with them)

Purpose

- Next you will learn how to create, execute, and manage two types of PL/SQL subprograms that are named and stored in the database, resulting in several benefits such as shareability, better security, and faster performance
- Two types of subprograms:
  - Functions
  - Procedures

Differences Between Anonymous Blocks and Subprograms

- As the word “anonymous” indicates, anonymous blocks are unnamed executable PL/SQL blocks
- Because they are unnamed, they can neither be reused nor stored in the database for later use
- While you can store anonymous blocks on your PC, the database is not aware of them, so no one else can share them
- Procedures and functions are PL/SQL blocks that are named, and they are also known as subprograms

Remember, every object stored in the database – tables, views, indexes, synonyms, sequences, and now PL/SQL subprograms – must have a name.

Differences Between Anonymous Blocks and Subprograms

- These subprograms are compiled and stored in the database
- The block structure of the subprograms is similar to the structure of anonymous blocks
- While subprograms can be explicitly shared, the default is to make them private to the owner's schema
- Later subprograms become the building blocks of packages and triggers

Differences Between Anonymous Blocks and Subprograms

- Anonymous blocks

```
DECLARE    (Optional)
   Variables, cursors, etc.;
BEGIN      (Mandatory)
   SQL and PL/SQL statements;
EXCEPTION (Optional)
   WHEN exception-handling actions;
END; (Mandatory)
```

- Subprograms (procedures)

```
CREATE [OR REPLACE] PROCEDURE name [parameters] IS|AS (Mandatory)
  Variables, cursors, etc.; (Optional)
BEGIN      (Mandatory)
   SQL and PL/SQL statements;
EXCEPTION (Optional)
   WHEN exception-handling actions;
END [name]; (Mandatory)
```

Differences Between Anonymous Blocks and Subprograms

- The alternative to an anonymous block is a named block. How the block is named depends on what you are creating
- You can create :
  - a named procedure (does not return values except as out parameters)
  - a function (must return a single value not including out parameters)
  - a package (groups functions and procedures together)
  - a trigger

Differences Between Anonymous Blocks and Subprograms

- The keyword DECLARE is replaced by CREATE PROCEDURE procedure-name IS | AS
- In anonymous blocks, DECLARE states, "this is the start of a block"
- Because CREATE PROCEDURE states, "this is the start of a subprogram," we do not need (and must not use) DECLARE

Differences Between Anonymous Blocks and Subprograms

Anonymous Blocks	Subprograms
Unnamed PL/SQL blocks	Named PL/SQL blocks
Compiled on every execution	Compiled only once, when created
Not stored in the database	Stored in the database
Cannot be invoked by other applications	They are named and therefore can be invoked by other applications
Do not return values	Subprograms called functions must return values
Cannot take parameters	Can take parameters

Almost every programming language supports subprograms. They are a part of structured programming. We can consider it as a separate block of statements (with its own declarations and programming statements), but still under the control of the main program.

The main block calls the subprogram by its name to execute its set of statements. This may be a part of the conditions as well. That means the main program may or may not call subprogram based on certain conditions.

Repeatable tasks are generally separated as subprograms to allow the user to use them as many times as possible. This also improves the readability of the main program, which is being divided into several meaningful tasks, where each task (subprogram) may have its own set of programming statements (including local declarations).

Differences Between Anonymous Blocks and Subprograms

- The keyword DECLARE is replaced by CREATE PROCEDURE procedure-name IS | AS
- In anonymous blocks, DECLARE states, "this is the start of a block"
- Because CREATE PROCEDURE states, "this is the start of a subprogram," we do not need (and must not use) DECLARE

Benefits of Subprograms

- Procedures and functions have many benefits due to the modularizing of the code:
  - Easy maintenance: Modifications need only be done once to improve multiple applications and minimize testing
  - Code reuse: Subprograms are located in one place
- When compiled and validated, they can be used and reused in any number of applications

Benefits of Subprograms

- Improved data security: Indirect access to database objects is permitted by the granting of security privileges on the subprograms
- By default, subprograms run with the privileges of the subprogram owner, not the privileges of the user
- Data integrity: Related actions can be grouped into a block and are performed together ("Statement Processed") or not at all

Subprogram security and privileges will be covered in a later section. For now, if a subprogram contains a SQL statement which SELECTs from a table, the subprogram owner needs SELECT privileges on that table, but the user does not. This means that the only way the user can access the table is through the subprogram, not by any other route. Therefore the user cannot see sensitive or confidential data unless the subprogram code explicitly allows it.

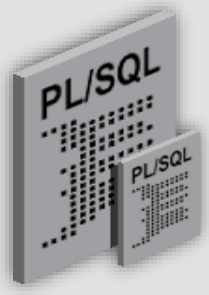
Benefits of Subprograms

- Improved performance: You can reuse compiled PL/SQL code that is stored in the shared SQL area cache of the server
- Subsequent calls to the subprogram avoid compiling the code again
- Also, many users can share a single copy of the subprogram code in memory
- Improved code clarity: By using appropriate names and conventions to describe the action of the routines, you can reduce the need for comments, and enhance the clarity of the code

Improved performance: a subprogram is compiled only once, when it is (re)CREATEd. An anonymous block is compiled (while the user waits) every time it is executed.

Procedures and Functions : Similarities

- Are named PL/SQL blocks
- Are called PL/SQL subprograms
- Have block structures similar to anonymous blocks:
  - Optional parameters
  - Optional declarative section (but the DECLARE keyword changes to IS or AS)
  - Mandatory executable section
  - Optional section to handle exceptions
- Procedures and functions can both return data as OUT and IN OUT parameters



Packages will be covered in a later section. For now, a package is a group of several related procedures and/or functions which are created and managed as a single unit. Packages are very powerful and important, but the "building blocks" are still procedures and functions.

Procedures and Functions : Differences

- A function MUST return a value using the RETURN statement
- A procedure can only return a value using an OUT or an IN OUT parameter
- The return statement in a function returns control to the calling program and returns the results of the function
- The return statement within a procedure is optional
- It returns control to the calling program before all of the procedure's code has been executed
- Functions can be called from SQL, procedures cannot
- Functions are considered expressions, procedures are not

What Is a Procedure?

- A procedure is a named PL/SQL block that can accept parameters
- Generally, you use a procedure to perform an action (sometimes called a “side-effect”)
- A procedure is compiled and stored in the database as a schema object
  - Shows up in USER_OBJECTS as an object type of PROCEDURE
  - More details in USER_PROCEDURES
  - Detailed PL/SQL code in USER_SOURCE

Syntax for Creating Procedures

- Parameters are optional
- Mode defaults to IN
- Datatype can be either explicit (for example, VARCHAR2) or implicit with %TYPE
- Body is the same as an anonymous block

```
CREATE [OR REPLACE] PROCEDURE procedure_name
 [(parameter1 [mode1] datatype1,
   parameter2 [mode2] datatype2,
  . . .)]
IS|AS
procedure_body;
```


There is no difference between IS and AS, either can be used.

An option of the CREATE PROCEDURE statement is to follow CREATE by OR REPLACE. The advantage of doing so is that should you have already created the procedure in the database, you will not get an error. On the other hand, should the previous definition be a different procedure of the same name, you will not be warned, and the old procedure will be lost.

There can be any number of parameters, each followed by a mode and a type. The modes are IN (read-only), OUT (write-only), and IN OUT (read and write). You will learn more about parameters and their modes in the next two lessons.

Syntax for Creating Procedures

- Use CREATE PROCEDURE followed by the name, optional parameters, and keyword IS or AS
- Add the OR REPLACE option to overwrite an existing procedure
- Write a PL/SQL block containing local variables, a BEGIN, and an END (or END procedure_name)

```
CREATE [OR REPLACE] PROCEDURE procedure_name
 [(parameter1 [mode] datatype1,
   parameter2 [mode] datatype2, ...)]
IS|AS
  [local_variable_declarations; …]
BEGIN
  -- actions;
END [procedure_name];
```

PL/SQL Block

Procedure: Example

- In the following example, the add_dept procedure inserts a new department with the department_id 280 and department_name ST-Curriculum
- The procedure declares two variables, v_dept_id and v_dept_name, in the declarative section

```
CREATE OR REPLACE PROCEDURE add_dept IS
  v_dept_id    dept.department_id%TYPE;
  v_dept_name  dept.department_name%TYPE;
BEGIN
  v_dept_id   := 280;
  v_dept_name := 'ST-Curriculum';
  INSERT INTO dept(department_id, department_name)
    VALUES(v_dept_id, v_dept_name);
  DBMS_OUTPUT.PUT_LINE('Inserted '|| SQL%ROWCOUNT || ' row.');
END;
```

- The declarative section of a procedure starts immediately after the procedure declaration and does not begin with the keyword DECLARE
- This procedure uses the SQL%ROWCOUNT cursor attribute to check if the row was successfully inserted
- SQL%ROWCOUNT should return 1 in this case

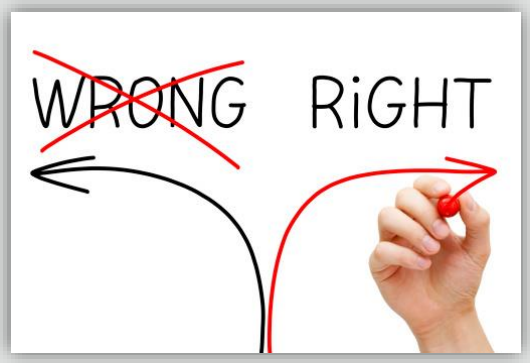
In a real-life procedure, the v_dept_id and v_dept_name values would be passed into the procedure as parameters, not hard-coded as shown here.

Procedure: Example

```
CREATE OR REPLACE PROCEDURE add_dept IS
  v_dept_id    dept.department_id%TYPE;
  v_dept_name  dept.department_name%TYPE;
BEGIN
  v_dept_id   := 280;
  v_dept_name := 'ST-Curriculum';
  INSERT INTO dept(department_id, department_name)
    VALUES(v_dept_id, v_dept_name);
  DBMS_OUTPUT.PUT_LINE('Inserted '|| SQL%ROWCOUNT || '
row.');
END;
```

Invoking Procedures

- You can invoke (execute) a procedure from:
  - An anonymous block
  - Another procedure
  - A calling application
- Note: You cannot invoke a procedure from inside a SQL statement such as SELECT



Invoking the Procedure from Application Express

- To invoke (execute) a procedure in Oracle Application Express, write and run a small anonymous block that invokes the procedure
- For example:

```
BEGIN
  add_dept;
END;

SELECT department_id, department_name FROM dept WHERE
department_id=280;
```

- The select statement at the end confirms that the row was successfully inserted

Correcting Errors in CREATE PROCEDURE Statements

- If compilation errors exist, Application Express displays them in the output portion of the SQL Commands window
- You must edit the source code to make corrections
- When a subprogram is CREATEd, the source code is stored in the database even if compilation errors occurred

Correcting Errors in CREATE PROCEDURE Statements

- After you have corrected the error in the code, you need to recreate the procedure
- There are two ways to do this:
  - Use a CREATE OR REPLACE PROCEDURE statement to overwrite the existing code (most common)
  - DROP the procedure first and then execute the CREATE PROCEDURE statement (less common)

Saving Your Work

- Once a procedure has been created successfully, you should save its definition in case you need to modify the code later

```
CREATE OR REPLACE PROCEDURE add_dept IS
   v_dept_id    dept.department_id%TYPE;
   v_dept_name  dept.department_name%TYPE;
BEGIN
   v_dept_id :=280;
   v_dept_name := 'ST-Curriculum';
   INSERT INTO dept(department_id, department_name)
      VALUES(v_dept_id, v_dept_name);
   DBMS_OUTPUT.PUT_LINE('Inserted '|| SQL%ROWCOUNT || ' row');
END;
```

Results | Explain | Describe | Saved SQL | History

```
Procedure created.
```

Saving Your Work

- In the Application Express SQL Commands window, click the SAVE button, then enter a name and optional description for your code
- You can view and reload your code later by clicking on the Saved SQL button in the SQL Commands window

Local Subprograms

- When one procedure invokes another procedure, we would normally create them separately, but we can create them together as a single procedure if we like

```
CREATE OR REPLACE PROCEDURE subproc
  ...
END subproc;

CREATE OR REPLACE PROCEDURE mainproc
  ...
IS BEGIN
  ...
    subproc(...);
  ...
END mainproc;
```

Local Subprograms

- All the code is now in one place, and is easier to read and maintain
- The nested subprogram's scope is limited to the procedure within which it is defined; SUBPROC can be invoked from MAINPROC, but from nowhere else

```
CREATE OR REPLACE PROCEDURE mainproc
  ...
IS
  PROCEDURE subproc (...) IS BEGIN
    ...
  END subproc;
BEGIN
  ...
    subproc(...);
  ...
END mainproc;
```

ORACLE Academy

This is similar to the idea of nested anonymous blocks covered earlier in the course. The example shown here shows nested procedures, but PL/SQL functions can also be nested in this way.

Why don't we just write all the code in a single (normal) procedure?

Answer: to restrict the scope of variables. Normal scoping rules apply, so a variable declared within the nested subprogram cannot be referenced in the outer program. This helps to make it clear which variables are used in which part of the code.

Local Subprograms

- Every time an employee is deleted, we need to insert a row into a logging table
- The nested procedure LOG_EMP is called a Local Subprogram

Local Subprograms

```
CREATE OR REPLACE PROCEDURE delete_emp
  (p_emp_id IN employees.employee_id%TYPE)
IS
  PROCEDURE log_emp (p_emp IN employees.employee_id%TYPE)
  IS BEGIN
    INSERT INTO logging_table VALUES(p_emp, ...);
  END log_emp;
BEGIN
  DELETE FROM employees
    WHERE employee_id = p_emp_id;
  log_emp(p_emp_id);
END delete_emp;
```

Alternative Tools for Developing Procedures

- If you end up writing PL/SQL procedures for a living, there are other free tools that can make this process easier
- For instance, Oracle tools, such as SQL Developer and JDeveloper assist you by:
  - Color-coding commands vs variables vs constants
  - Highlighting matched and mismatched (parentheses)
  - Displaying errors more graphically

Alternative Tools for Developing Procedures

- Enhancing code with standard indentations and capitalization
- Completing commands when typing
- Completing column names from tables

Alternative Tools for Developing Procedures

- To develop a stored procedure when not using Oracle Application Express, perform the following steps:
  - 1. Write the code to create a procedure in an editor or a word processor, and then save it as a SQL script file (typically with a .sql extension)
  - 2. Load the code into one of the development tools such as iSQL*Plus or SQL Developer
  - 3. Create the procedure in the database
  - The CREATE PROCEDURE statement compiles and stores source code and the compiled m-code in the database
  - If compilation errors exist, then the m-code is not stored and you must edit the source code to make corrections

Alternative Tools for Developing Procedures

- To develop a stored procedure when not using Oracle Application Express, perform the following steps:
  - 4. After successful compilation, execute the procedure to perform the desired action
  - Use the EXECUTE command from iSQL*Plus or an anonymous PL/SQL block from environments that support PL/SQL

Terminology

- Key terms used in this lesson included:
  - Anonymous blocks
  - IS or AS
  - Procedures
  - Subprograms

ORACLE Academy

- Anonymous Blocks – Unnamed executable PL/SQL blocks that can not be reused no stored for later use.
- IS or AS – Indicates the DECLARE section of a subprogram.
- Procedures – Named PL/SQL blocks that can accept parameters and are compiled and stored in the database.
- Subprograms – Named PL/SQL blocks that are compiled and stored in the database.

Summary

- In this lesson, you should have learned how to:
  - Differentiate between anonymous blocks and subprograms
  - Identify the benefits of subprograms
  - Define a stored procedure
  - Create a procedure
  - Describe how a stored procedure is invoked
  - List the development steps for creating a procedure
  - Create a nested subprogram in the declarative section of a procedure


ORACLE
Academy